

大数据的数学基础

Consistent Hashing

Consistent Hashing

hash ring

$$0 - 2^{32} - 1$$

0在环的正上方，顺时针围成一圈

confirm location

server

选择服务器的 IP 地址或主机名作为密钥，分别对服务器的密钥执行哈希运算

确定每个服务器在哈希环中的位置

data storage

使用相同的函数来计算键的哈希值

按顺时针方向搜索，并将数据存入遇到的第一个服务器

data server failure

在一致性哈希算法中，如果某台服务器不可用，唯一受影响的数据仅限于该服务器与其环空间中上一台服务器之间的数据（在此例中，即 C 和 B 之间的数据），其余数据则不受影响

Server X is added to the system

在一致性哈希算法中，当节点被添加或移除时，环形空间中的数据只需进行少量重新定位。

一致性哈希和主从备份MS-replication 结合

一致性哈希负责决定数据应该放在哪个服务器；主从复制负责在服务器挂掉时提供备份；如果主服务器失败，就切换到它的备份节点；如果新增服务器，新的写入直接写到新节点，旧数据可以通过顺时针查找和懒迁移逐步搬到新节点。

Data skewing 数据倾斜

当一致性哈希环上的服务器节点太少或分布不均匀时，数据可能集中到少数服务器上，造成数据倾斜。解决办法是使用虚拟节点：让一个物理服务器对应多个 hash 位置，使服务器在环上分布更均匀，最后再把虚拟节点映射回真实服务器。

expected load

一致性哈希中，M 个对象分布到 N 台服务器上，每台服务器期望负载是 M/N 。新增服务器时，只需要迁移新服务器负责区间内的大约 M/N 个对象，而不是重新分配所有对象。Lookup 和 Insert 都可以看成是在哈希环上从对象位置开始顺时针寻找第一个服务器节点。一致性哈希不能完全避免 hash 冲突，但可以大幅减少服务器增删时的数据迁移成本。

Hash conflicts

用链表或红黑树处理哈希冲突

chain table

时间复杂度一般为 $O(n)$

red-black tree 红黑树

当链表过长时，考虑红黑树

- 节点为红色或黑色
- 根节点为黑色
- 每个叶节点都是一个黑色空节点（NULL节点）
- 每个红色节点的两个子节点都是黑色（不允许出现两个连续的红色节点）
- 从任意一个节点到其每个叶节点的所有路径中，包含的黑色节点数量均相同

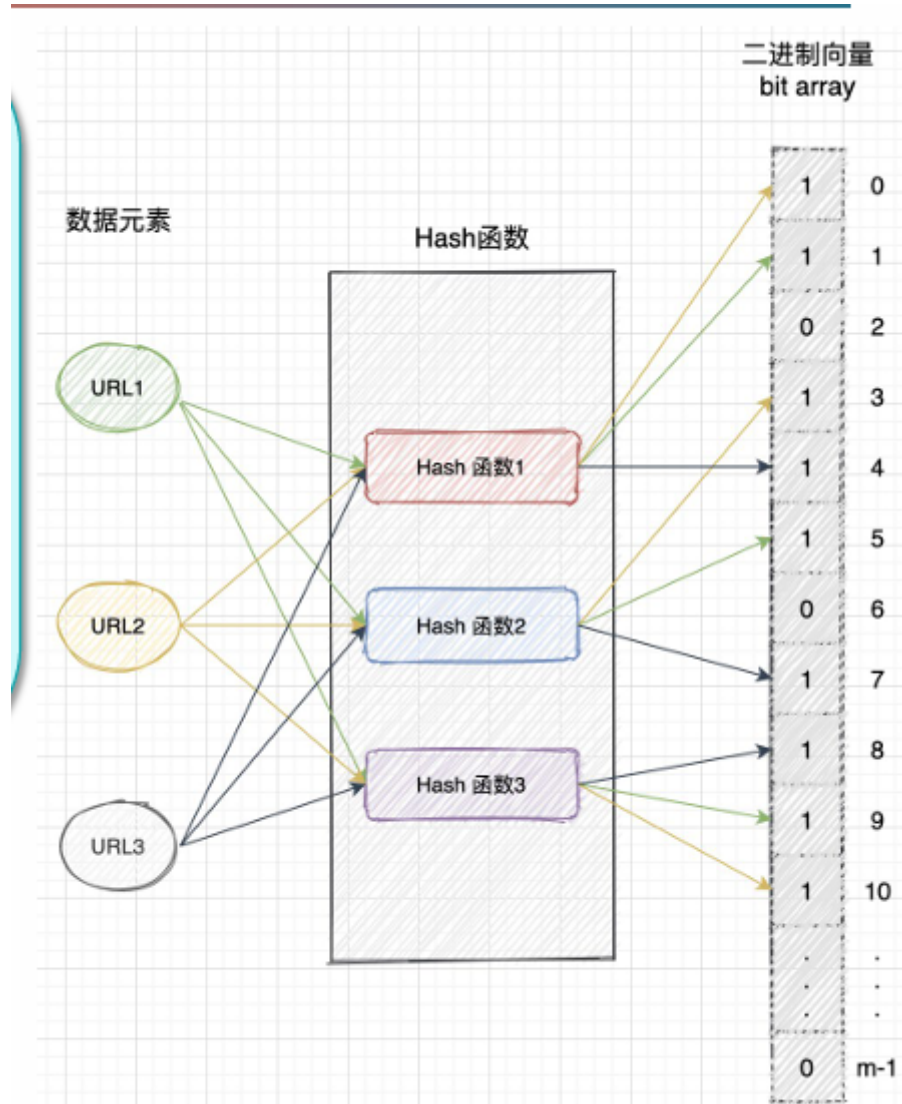
n个内部节点的红黑树的高度h满足 $h \leq 2 \log(n + 1)$ ，复杂度 $O(\log n)$

Bloom Fliter

用哈希表存储时会消耗大量空间，而且可能存在哈希冲突

布隆过滤器在大数据去重以及判断数据是否存在方面有着广泛的应用。它是一种空间效率高的概率算法和数据结构，用于判断元素是否属于集合，其核心是一个非常长的二进制向量和一系列哈希函数。

原理：当向集合中添加一个元素时，该元素会被映射到K个哈希函数映射到位数组中的K个点，并将这些点设为1。在检索时，我们只需检查这些点是否全为1，即可（近似地）判断该元素是否在集合中：若其中任何一点为0，则被检查的元素必定不在集合中；若所有点均为1，则被检查的元素很可能存在于集合中。



判定一个元素不存在，那么它肯定不存在

但是如果bloom filter判断一个元素存在,那么它有可能判断错了，即元素可能不存在

优点

节省空间，查询无需遍历，只需取数组脚标

缺点

会造成误判，而且无法删除元素

False Positives 可能误判的概率

m 是数组的位数， k 是哈希函数的个数， n 是元素的个数,那么数组中某位为0的概率是：

$$(1 - 1/m)^{kn}$$

那么一个元素误判它存在的概率为:(判断元素为1了 k 次)

$$(1 - (1 - 1/m)^{kn})^k$$

由

$$\lim(1 - 1/m)^m = 1/e$$

得到误判的概率约为

$$(1 - e^{-kn/m})^k$$

当 $k = \ln 2 * m/n$ 时，误判率最小(对上面的 $f(k)$ 求导，求极值)

进而可以求出 m

Heavy Hitters

更新元素在 HashMap 的频率：不存在就添加，存在就把次数加一

然后更新 Heap：把堆中元素更新数值，调整堆；把不在堆中元素和堆顶元素比较，然后再调整堆

空间复杂度：

- HashMap: $O(n)$ ，存储所有元素
- Heap: $O(k), k \ll n$.

总体的空间复杂度为 $O(n)$.

时间复杂度：

- HashMap: $O(1)$
- Heap: $O(k + \log k)$ (搜索+调整)

总体 $O(n(1+k+\log k)) = O(n)$

Count-Min Sketch

不用存下所有元素，只用一个较小的二维数组，**近似估计每个元素出现了多少次**。

$d \times w$ 二维数组和 d 个哈希函数，添加元素时，正常计算 d 次哈希，然后在数组对应位置加1。

查询某元素出现频率时，计算 d 次哈希值，然后在其中取最小值。

空间复杂度： $O(d \times w)$

时间复杂度： $O(n)$

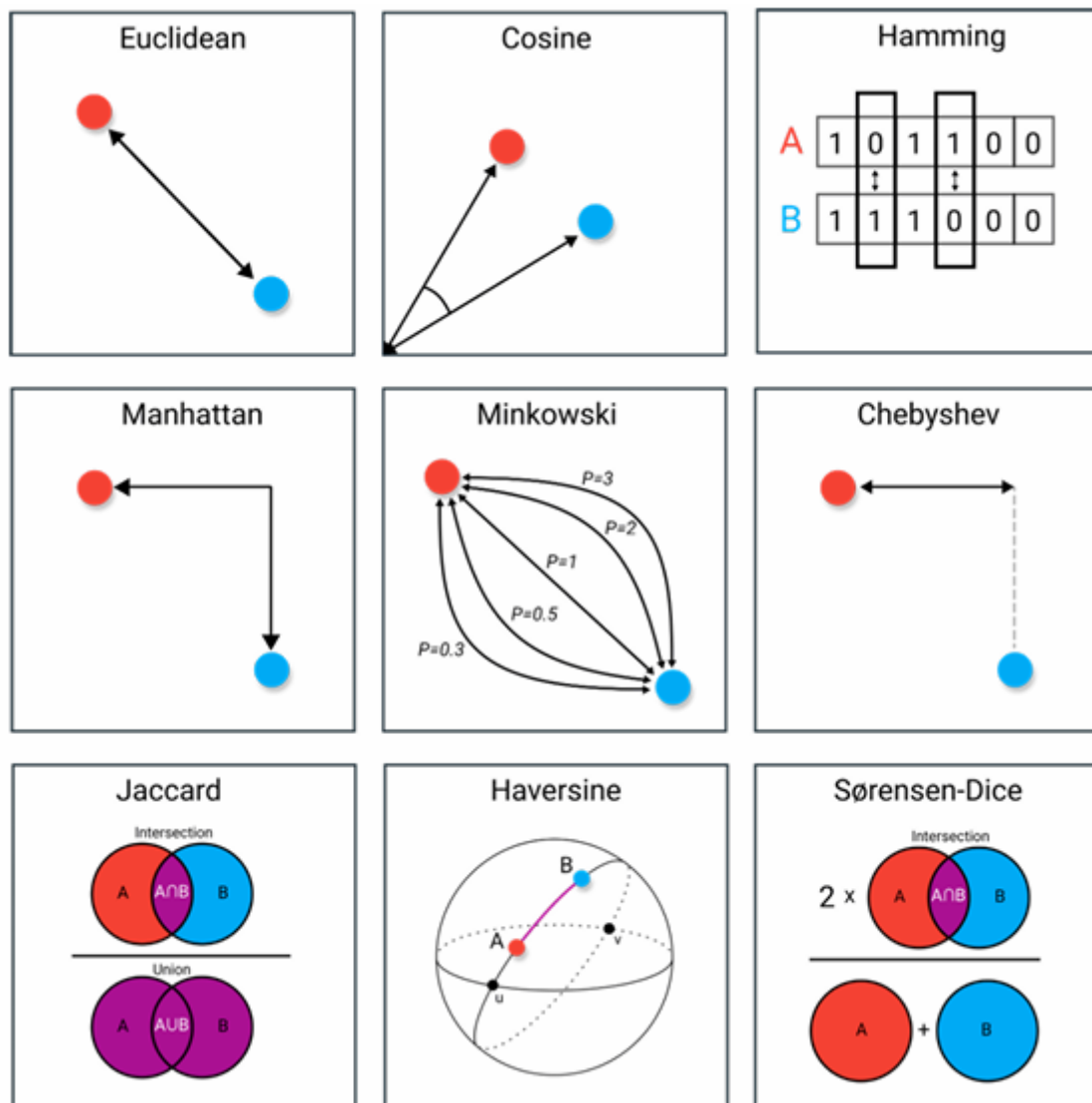
由于哈希冲突只会导致计数器偏大，因此 Count-Min Sketch 一般不会低估，只会准确或高估。宽度 w 控制误差大小， w 越大冲突越少；深度 d 控制失败概率， d 越大结果越可靠。

Conservative Update

更新时只在 d 个哈希值中最小的加一，可以避免 overestimation

ch2

Similarity Search



Jaccard Similarity

$J(S, T)$ = 共同元素数量除以总共元素的数量 in text

$$J(S, T) = \frac{|S \cap T|}{|S \cup T|}$$

Jaccard distance: $1 - J(S, T)$

如果两个对象可以表示成01向量，可以用其向量形式

$$J(A, B) = \frac{N_{A_1 B_1}}{N_{A_1 B_0} + N_{A_0 B_1} + N_{A_1 B_1}}$$

$N_{A_0 B_0}$ 经常忽略因为这个数很大，不一定说明二者相似

如果一个文档中，元素多次重复出现，需要用多重集合版

$$J(v_S, v_T) = \frac{\sum_i \min(v_S(i), v_T(i))}{\sum_i \max(v_S(i), v_T(i))}$$

比如看两篇文章的相似性，现在知道每个文章每个单词出现的频率，然后分别计算每个单词最小频率的和还要最大频率的和，最后作商得到相似度

Cosine Similarity(Distance)

看的是两个向量的方向接近程度，上一个关注的是有没有共同元素

在一句话相似度分析中，余弦会考虑词频，但是Jaccard不会

2.3 Norm 范数

性质

- 大小大于等于0，非负
- $\|cx\| = |c|\|x\|$
- 满足三角不等式

常见范数

L_0 norm: $\|x\|_0$ = 向量中非零元素的个数

L_1 norm: $\|x\|_1 = \sum_i |x_i|$, 所有元素绝对值之和

(曼哈顿距离Manhattan: 只能横竖走，不能斜着穿过去)

L_2 norm: $\|x\|_2 = \sqrt{\sum_i x_i^2}$, 平方和开根号, (欧氏距离Euclidean)

L_∞ norm: $\|x\|_\infty = \max_i |x_i|$, 取向量中元素绝对值最大(切比雪夫距离Chebyshev)

L_p norm: $\|x\|_p = (\sum_{i=1}^n |x_i|^p)^{\frac{1}{p}}$, 是以上的统一形式(Minkowski闵可夫斯基)

Matrix Norm 矩阵范数

非负，齐次，三角不等式

相容

如果矩阵范数和向量范数满足

$$\|Ax\|_v \leq \|A\|_m \|x\|_v$$

那么它们是compatible

常见矩阵范数

$\|A\|_{m1}$ 把所有元素取绝对值，然后相加

$$\|A\|_F = \sqrt{\sum_i \sum_j |a_{ij}|^2} \text{ 所有元素平方和开根}$$

$\|A\|_M$: 最大元素绝对值乘 $\max\{m, n\}$ (矩阵 m 行 n 列)

$\|A\|_{m\infty}$: 最大元素绝对值乘列数

$\|A\|_G = \sqrt{mn}$ 乘最大元素绝对值

$$\|A\|_1 = \max_j \sum_i |a_{ij}| \text{ 每列绝对值和取最大}$$

$$\|A\|_\infty = \max_i \sum_j |a_{ij}| \text{ 每行绝对值和取最大}$$

$$\|A\|_2 = \sqrt{\lambda_{\max}(A^H A)}$$

2.4 K-d tree

KNN算法: 找距离最近的 K 个点，用少数服从多数原则分类，但是当数据规模很大时就会变慢

先把空间分区，查询时只去可能包含最近点的区域，不必检查所有点

构建步骤

1. 计算每个维度的方差，选择方差最大的维度作为初始划分轴
2. 在当前轴上找到中位数，作为当前树的节点
3. 然后按照这个数，分成左右子树
4. 循环使用维度 $r=(r+1)\%k$,重复以上过程

最近邻搜索

给定树和查询点x，输出距离最近的M个点

从根向下，如果查询点在该维度的坐标小于当前点，就进入左子树，否则进入右子树，直到进入子节点

先把这个叶节点作为最近点计算距离，然后向上回溯，计算每个点到查询点的距离，检查另一侧子树是否有更近点，实时更新

什么时候检查另一侧子树：检查这一维度的距离d和当前最近距离r的大小

维度越高效果越差

BBF

为了解决大量回溯导致性能下降的问题

用优先队列优先搜索最有可能包含近邻的区域，同时设置超时或搜索上限，得到近似最近邻

2.5 Curse of Dimensionality 维度灾难

随着维度增加，空间急剧膨胀，样本变得稀疏，模型容易过拟合，算法效率下降

真正的目标应该是找到合适数量的有效特征

Dimensionality Reduction

用更少的维度保留主要信息

常见方法：

- Feature selection: 特征选择
- Feature Transformation: 特征变换

2.6 Minhash

近似估计Jaccard相似度

构造特征矩阵，行是元素，列是每个集合

打乱矩阵的行，从每个列从上往下找第一个1，这个1所在的行号就是这个集合的MinHash值

$$P(h(S_1) = h(S_2)) = J(S_1, S_2)$$

Signature Matrix

做多次随机排列，得到多个MinHash的值，每个集合得到n个，组成一个列向量叫做这个集合的signature

$$SIM(S_i, S_j) = \frac{\text{两个签名中相等的位置数}}{\text{签名总长度}}$$

ch3

3.1 什么是泛化 generalization

Optimization: 让模型在训练集上表现好, 降低训练误差

Generalization: 模型在没见过的新数据表现也好, 降低泛化误差

机器学习的难点就在于: **如何通过训练集上的学习, 得到一个对新数据也有效的模型**

- underfitting 欠拟合: 模型太简单, 没学到规律
- fitting 正常拟合: 理想状态
- overfitting 过拟合: 把细节和噪声记住了
- no restraint 无约束: 学习过程没有被约束, 结果完全不可靠

1. **Error rate:** 一共有 m 个样本, 其中 a 个样本分类错了, 错误率 $E = \frac{a}{m}$

2. **Accuracy:** $1 - \frac{a}{m}$

3. **Error:** 预测输出和真实输出的差 difference

4. **Training/ empirical error 训练/经验误差:** 模型在训练集的错误率

5. **Test error 测试误差:** 模型在测试集的错误率

6. **Generalization error 泛化误差:** 模型在未知数据的期望错误率

测试集不能参与训练, 并且训练集和测试集要来自同一个真实分布, 不能分布偏移 distribution shift, 用测试误差近似泛化误差

3.2 如何理解泛化

泛化能力: 模型对新数据分类的表现

Adversarial Validation: 把训练集样本标记为 0, 测试集样本标记为 1, 然后训练一个分类器, 看它能不能分出“这个样本来自训练集还是测试集”, 如果很容易分出来就说明他们的分布不一致

Sparsity 稀疏性: 泛化能力强的模型, 通常应该更简单、更稀疏。稀疏指的是模型参数中有很多接近 0, 甚至等于 0。模型只保留真正重要的特征, 忽略不重要的特征。参数越小模型越简单, 更容易泛化

具有泛化能力的模型应该能够在每个抽象层级重建 (reconstruct) 重要特征。也就是说, 模型不仅能输出结果, 还能在内部捕获并保留数据的关键结构信息。

模型可以有效忽略无关或琐碎特征, 或者在不相关的变化下仍能识别相同特征。

Dropout 是深度学习中常用的一种正则化方法, 通过随机失活神经元, 降低过拟合, 提高泛化能力。

泛化能力也可以理解为信息压缩能力

高泛化能力的模型能够最小化在真实环境中遭遇意外情况的风险

K-fold Cross Validation K折交叉验证

把原始训练数据分成 K 份, 每次拿其中 1 份做验证集, 剩下 $K-1$ 份做训练集。重复 K 次, 让每一份都当一次验证集, 最后把 K 次结果取平均。

K 较大, 偏差 Bias 较小, 方差 Variance 较大

Bias、Variance、Noise

$$Error = Bias + Variance + Noise$$

- **Bias error:** 模型预测结果和真实结果之间的系统性差距。偏差高，说明模型本身太简单、不够准确，容易欠拟合。
- **Variance error:** 模型对不同训练数据的敏感程度。方差高，说明模型不稳定，训练集稍微变一点，模型结果就变很多，容易过拟合。
- **Noise:** 数据本身的随机性或错误，比如标注错误、测量误差、异常点。

模型越简单，方差小但偏差大；模型越复杂，偏差小但方差大。我们要找的是二者之间的平衡点。

KL Divergence 散度

$$D_{KL}(P \parallel Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)} (\text{nats})$$

用来衡量两个概率分布之间的差异，真实分布是 P ，近似分布是 Q ，那么表示用 Q 近似真实分布 P ，平均会损失多少信息

特点：

- **Asymmetry 不对称性:** $D_{KL}(P \parallel Q) \neq D_{KL}(Q \parallel P)$
- **Non-negativity 非负:** $D_{KL}(P \parallel Q) \geq 0$
- **Not a metric:** 这违反了三角不等式，因此并非一种真正的“距离”度量。

ERM 和 SRM：正则化的核心思想

ERM 经验风险最小化

只关心训练模型损失最小，但是可能导致过拟合

SRM 结构风险最小化

不仅训练误差小，还要模型复杂度低

SRM等价于正则化

3.3 Dataset classification

训练集 training set: 平时刷的练习题，用来训练模型。

验证集 validation set: 模拟考试题，用来调参数、选模型。

测试集 test set: 正式考试题，只能最后用一次，用来评估模型真实水平。

测试集绝对不能参与任何训练和调参

关于机器学习

$$Learning = Representation + Evaluation + Optimization$$

Representation 表示

表示模型的形式，这个选择决定了模型的 **假设空间 hypothesis space**

Evaluation 评价

判断模型好坏的指标

Optimization 优化

找到最优模型的方法

- **线性回归**：表示是线性方程；评价通常用 MSE；优化常用梯度下降。
- **逻辑回归**：表示是线性函数加 sigmoid 概率映射；评价用交叉熵；优化用梯度下降或牛顿法。
- **决策树**：表示是树结构；评价用信息增益或 Gini 指数；优化通常是贪心选择最优划分特征。
- **支持向量机 SVM**：表示是分类超平面；评价目标是最大化间隔 margin；优化是凸二次规划。
- **神经网络**：表示是多层非线性网络；评价用交叉熵或 MSE；优化用反向传播加 SGD 或其他优化器。

混淆矩阵

情况	含义
TP	True Positive, 真是正类, 预测也是正类
TN	True Negative, 真是负类, 预测也是负类
FP	False Positive, 真是负类, 但预测成正类
FN	False Negative, 真是正类, 但预测成负类

$$Accuracy(acc) = \frac{TP + TN}{TP + TN + FP + FN}$$

$$Error(err) = \frac{FP + FN}{TP + TN + FP + FN}$$

Precision 查准率:

$$P = \frac{TP}{TP + FP}$$

Recall 查全率:

$$R = \frac{TP}{TP + FN}$$

F1值:

$$F_1 = \frac{2PR}{P + R}$$

F β 值:

$$F_\beta = \frac{(1 + \beta^2)PR}{\beta^2 P + R}$$

当 $\beta = 1$ 时, 就是 F1。

当 $\beta < 1$ 时, 更重视 Precision。

当 $\beta > 1$ 时, 更重视 Recall。

P-R 曲线就是 Precision-Recall 曲线, 如果一个模型的 P-R 曲线完全包住另一个模型, 说明它整体更好。**BEP Balance-Equal Point**, 即 Precision 和 Recall 相等时的点。

ROC & AUC

很多二分类输出不是0/1，而是一个概率，这需要认为设置一个阈值threshold，概率大于阈值，判为1，否则判为0

ROC曲线

评价模型在不同阈值下的整体分类能力

纵轴是TPR，横轴是FPR

TPR: 原来为正的样本有多少判断对的

$$TPR = \frac{TP}{TP + FN}$$

FPR: 原来为负的样本有多少判断错了

$$FPR = \frac{FP}{FP + TN}$$

绘制：先把所有预测分数从高到低排序，把阈值设很高，使所有样本预测为负，此时 $TPR = FPR = 0$ 。（从点(0,0)开始）然后逐个降低阈值，让样本一个个变成预测正类。

如果这个预测正类是真实的正类，TPR上升 $\frac{1}{m}$ ，m是正样本的数量，反之FPR上升 $\frac{1}{n}$ ，n是负样本数量，连点成线。

曲线越接近左上角越好， $TPR = 1, FPR = 0$ ，意味着所有的正类都找出来了也没有误报。

如果A的ROC曲线完全包住B的，说明A模型更好，如果两条曲线交叉则无法判断，这时看AUC

AUC area under curve

AUC是ROC曲线下的面积，面积越大代表整体分类能力越强

3.4 PAC Probably Approximately Correct 概率近似正确

理解

近似正确：模型的泛化误差足够小。设模型 h 的泛化误差是 $R(h)$ ，如果 $R(h) \leq \epsilon$ ，就说这个模型是近似正确的。

概率： $P(R(h) \leq \epsilon) \geq 1 - \delta$ ，其中 $1 - \delta$ 是置信度confidence

整体就是学习算法至少有 $1 - \delta$ 的概率学到一个泛化误差不超过 ϵ 的模型

一些数学概念

sample set X & label set Y

X是样本空间，是有可能输入，Y就是输出的标签，比如{0,1}

concept c

$$c(x) = y$$

c是一个完美的规则，可以对所有输入给出正确的标签，但是我们并不知道这个c，只能训练样本去逼近它

concept class C

是一组可能的正确规则

hypothesis space H

是学习算法能够选择的模型集合，由你的选择决定，H和C不一定相同

sample set S

$$S = \{(x_1, c(x_1)), (x_2, c(x_2)), \dots, (x_m, c(x_m))\}$$

每个样本的标签由真实概念给出，训练标签的任务是从S中找出一个假设 $h_s \in H$ ，使他尽量接近真实概念c

i.i.d.

independently and identically distributed 独立同分布是训练样本和测试样本必须满足的条件

$R(h)$ & $\hat{R}(h)$

泛化误差 generalization error/true error :

$$R(h) = P_{x \sim D}[h(x) \neq c(x)]$$

从真实分布D中随机抽一个样本，模型h判断错误的概率

期望形式：

$$R(h) = E_{x \sim D}[1_{h(x) \neq c(x)}]$$

$1_{h(x) \neq c(x)}$ 是指示函数，预测错了等于1，对了等于0。

通常无法直接计算，因为真实分布未知。

经验误差 empirical error :

$$\hat{R}(h) = \frac{1}{m} \sum_{i=1}^m 1_{h(x_i) \neq c(x_i)}$$

训练样本的错误率，比如100个样本，预测错了8个，那么这个值为0.08

两者关系：

$$E[\hat{R}(h)] = R(h)$$

在什么条件下PAC learnable

PAC identify 只说“能不能学到一个还不错的模型”

一个概念类 CCC 是 PAC 可学习的，意思是：存在一个学习算法，可以用合理数量的样本，在多项式时间内，以较高概率学到一个泛化误差足够小的模型。

sample complexity

为了达到指定误差 ϵ 和置信度 $1 - \delta$ ，至少需要多少训练样本

$$m \geq \text{poly}\left(\frac{1}{\epsilon}, \frac{1}{\delta}, \text{size}(x), \text{size}(c)\right)$$

如果你要求误差更小，置信度更高，输入更复杂、概念更复杂，需要更多样本。

distribution-free

不假设具体分布，但是必须独立同分布

3.5 ERM Empirical Risk Minimazation

机器学习中risk可以理解为模型预测结果和真实标签之间的平均误差

Loss function

某个样本真实值是 y_i ,模型预测是 $f(x_i)$,损失函数可写成:

$$L(y_i, f(x_i))$$

他只看一个样本

Empirical risk function

$$R_{emp}(f) = \frac{1}{N} \sum_{i=1}^N L(y_i, f(x_i))$$

所有训练样本损失的平均值

ERM就是找一个模型 f ,让训练集的平均损失最小

线性回归例子

线性回归用于预测连续值

只有一个自变量就是一元线性回归 Univariate, 多个自变量就是多元Multiple

Cost Function

单个样本的平方损失 : $L_i = (h_{\theta}(x_i) - y_i)^2$

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x_i) - y_i)^2$$

其中,

$$h_{\theta}(x_i) = \theta_0 x_0^{(i)} + \theta_1 x_1^{(i)} + \dots \theta_j x_j^{(i)}$$

h 是在参数 θ 下的预测函数, x_i 是第 i 个样本, 该样本一共有 $0-j$ 个特征

Least Squares Method 最小二乘法

$$L(w, b) = \sum_{i=1}^m (y_i - \hat{y}_i)^2 = \sum_{i=1}^m (y_i - wx_i - b)^2$$

目标是将上面的平方和最小化

对 b 求偏导, 然后让偏导为0, 得到 b , 然后用相同方法得到 w

$$-2 \sum_{i=1}^m (y_i - wx_i - b) = 0$$

$$b = \frac{1}{m} \sum_{i=1}^m (y_i - wx_i) = \bar{y} - w\bar{x}$$

$$w = \frac{\sum_{i=1}^m (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^m (x_i - \bar{x})^2}$$

对多元线性回归的矩阵公式

$$\hat{y} = w^T x + b$$

把 b 合并进参数，写成 $\hat{y} = X\hat{w}$

然后平方误差和可以写成： $L(\hat{w}) = (y - X\hat{w})^T (y - X\hat{w})$

对 w 求偏导： $\frac{\partial L(\hat{w})}{\partial \hat{w}} = 2X^T (X\hat{w} - y)$

令偏导 = 0： $\hat{w}^* = (X^T X)^{-1} X^T y$

Gradient Descent 梯度下降

数据量大之后最小二乘法效率低，常对平方损失代价函数用梯度下降

对第 j 个参数求偏导

$$\frac{\partial J(\theta)}{\partial \theta_j} = -\frac{1}{m} \sum_{i=1}^m (y_i - h_{\theta}(x_i)) x_j^{(i)}$$

然后对第 j 个参数进行更新：

$$\theta_j := \theta_j - \alpha \frac{\partial J(\theta)}{\partial \theta_j}$$

α 是学习率，多次迭代找到 empirical risk 最小的参数

Logistic Regression 逻辑回归

主要用于二分类

Sigmoid / Logistic Function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

输出范围在 (0,1) 之间，可以表示概率

Logistic Regression Model

$$P(Y = 1|X) = h_{\omega}(X) = \sigma(\omega^T X + b)$$

给定输入特征 X ，样本属于类别 1 的概率

Decision Boundary

输出概率后，由于是二分模型，需要一个 threshold，一般是 0.5

Loss Function

逻辑回归不用平方损失，而是从 **Maximum Likelihood Estimation, MLE 最大似然估计** 推导出 **log-loss 对数损失 / cross-entropy loss 交叉熵损失**

$$\text{条件概率: } P(Y|X; \omega) = h_{\omega}(X)^Y (1 - h_{\omega}(X))^{1-Y}$$

Likelihood 似然函数: $L(\omega) = \prod_{i=1}^n P(Y_i|X_i;\omega)$, 最大似然要把这个概率最大化

Log-Likelihood: $l(\omega) = \ln L(\omega)$ 逻辑回归要最大化, 也就是梯度上升, 机器学习习惯最小化loss, 所以令 $J(\omega) = -l(\omega)$, 这是总损失

如果写成平均损失, 就多除以一个样本数m

逻辑回归的梯度下降

对平均损失:

$$J(w) = -\frac{1}{m} \sum_{i=1}^m [Y_i \log h_i + (1 - Y_i) \log(1 - h_i)]$$

其中: $h_i = \sigma(w^T X_i + b) = \sigma(z_i)$, $\sigma'(z_i) = \sigma(z_i)(1 - \sigma(z_i))$

$$\text{所以 } \frac{\partial h_i}{\partial w_j} = h_i(1 - h_i)X_j^{(i)}$$

$$\text{再求 } \frac{\partial J_i}{\partial h_i} = -\left[\frac{Y_i}{h_i} - \frac{1 - Y_i}{1 - h_i}\right] = \frac{h_i - Y_i}{h_i(1 - h_i)}$$

$$\text{两项相乘得到 } \frac{\partial J_i}{\partial w_j} = (h_i - Y_i)X_j^{(i)}$$

这是第 i 个样本的 J , 然后把所有样本相加取平均:

$$\frac{\partial J}{\partial w_j} = \frac{1}{m} \sum_{i=1}^m (h_i - Y_i)X_j^{(i)}$$

同理, 对 b 求导, 除了 $\frac{\partial z_i}{\partial b} = 1$, 其他相同

$$\frac{\partial J}{\partial b} = \frac{1}{m} \sum_{i=1}^m (h_i - Y_i)$$

$$\text{最后更新参数: } w_j = w_j - \alpha \frac{\partial J}{\partial w_j}, b = b - \alpha \frac{\partial J}{\partial b}$$

3.6 SRM 结构风险最小化

因为只看ERM可能会导致过拟合, 所以介绍SRM

$$R_s(f) = \frac{1}{N} \sum_{i=1}^N L(y_i, f(x_i)) + \lambda J(f)$$

前半部分是经验风险ERM, 后面代表了模型复杂度, λ 是正则化系数, 越大越惩罚复杂模型

L1正则化

$$R_s(f) = \frac{1}{2m} \sum_{i=1}^m L(y_i, f(x_i)) + \frac{\lambda}{m} \sum_{j=1}^n |\theta_j|$$

L2正则化

$$R_s(f) = \frac{1}{2m} \sum_{i=1}^m L(y_i, f(x_i)) + \frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2$$

ch4 PCA主成分分析

高维数据太复杂, 我们想把它降到低维, 但尽量少丢信息

4.1 低维空间的数据可视化

降维分成两类：

- 特征选择：选择原特征的一部分保留，其余删掉
- 特征变换：把原特征组合成新的特征，**PCA属于这一种**，生成的新特征叫做主成分

线性降维：低维数据仍保持高维数据之间的线性关系，快而且复杂度低，**PCA, LDA, LPP**

非线性降维：保持数据的局部结构，**KPCA**

4.2 PCA

PCA 是一种**无监督学习的线性降维方法**

PCA	LDA
unsupervised learning 无监督	supervised learning 有监督
不用类别标签 label	需要类别标签 label
目标是最大化投影后方差	目标是类内差异小、类间差异大
用于降维、可视化、去冗余	常用于分类前的降维

4.3 Solving steps

假设有M个样本，N个特征，构成了M行N列的矩阵X

Remove the mean

对每个特征单独求均值，然后把每个样本的该特征减去平均值

$$X_c = X - \bar{X}$$

Covariance matrix 协方差矩阵

$$C = \frac{1}{M-1} X_c^T X_c$$

得到一个n*n矩阵，对角线元素是方差，非对角线元素是协方差

Compute eigenvalues and eigenvectors 特征值和特征向量

$$Cu = \lambda u$$

协方差矩阵C是实对称矩阵，满足性质：

不同特征值对应的特征向量互相正交

特征向量可以单位化

特征向量可以看成一组新的坐标轴

Sort eigenvalues 排序特征值

把特征值**从大到小**排序，对应的特征向量也按这个顺序排列

前k个特征向量就是要保留的k个主成分方向

为什么特征值越大排在越前面：

设投影的单位向量是 u ,投影后的所有样本方差是 $var = u^T C u$, PCA要找方差最大的方向, 这个方向恰好对应最大特征值。

保留方差大的方向, 是因为信号的方差较大, 噪音的方差较小。

Keep top k eigenvectors 保留前k个特征向量

把前k个特征向量组合成矩阵W, n行k列

Project 投影

$$Z = X_c W$$

Z是一个M行k列的矩阵, 这样把N个特征变成了k个特征

PCA中的数学原理

dot product 点积:

$$a * b = |a||b| \cos \theta$$

当b为单位向量时候, 那么这个点积就是a在b方向的投影长度

所以矩阵乘法等于批量做点积, $x_i^T u$ 表示第i个样本的投影

Basis: PCA在找一组新的坐标轴: 就是k个特征向量

为了使特征之间方差尽量大, 两两协方差为0

PCA的优化目标

把新数据的协方差矩阵变成对角线元素为特征值, 其他元素都是0

这是 diagonalization of covariance matrix **协方差矩阵对角化**

无偏估计

计算协方差除以 $M - 1$ 而不是 M

Explained Variance 解释方差

衡量前k个主成分保留了多少信息, 用解释方差比例计算前k个主成分累计贡献率

$$R_k = \frac{\sum_{i=1}^k \lambda_i}{\sum_{j=1}^N \lambda_j}$$

补充PCA的用途

降维, 数据可视化, 去除噪声, 去冗余, 数据压缩, 提取主要特征

ch5 SVD 奇异值分解

5.1 From PCA to SVD

Eigenvalue Decomposition 特征值分解:

$$A = P\Lambda P^{-1}$$

P是特征向量组成的矩阵， Λ 是特征值组成的对角矩阵，只有**方阵可以直接做特征值分解**

相比于特征值分解，**SVD可以处理任意实矩阵**

正交矩阵 Orthogonal Matrix(复数空间叫酉矩阵 Unitary)满足：

$$U^T = U^{-1}$$

正交变换和旋转：

左乘正交矩阵相当于做正交变换(旋转)，不改变长度

5.2 SVD

$$A = U \Sigma V^T$$

设： $A \in \mathbb{R}^{m \times n}$

左奇异向量矩阵： $U \in \mathbb{R}^{m \times m}$

奇异值矩阵： $\Sigma \in \mathbb{R}^{m \times n}$

右奇异向量矩阵： $V^T \in \mathbb{R}^{n \times n}$

先用 V^T 做旋转， Σ 进行缩放，再 U 旋转

进一步，可以推出：

$$A^T A = V(\Sigma)^2 V^T$$

$$A A^T = U(\Sigma)^2 U^T$$

所以 V 是来自 $A^T A$ 的特征向量矩阵， U 是 $A A^T$ 的特征向量矩阵

奇异值是特征值的平方根，这里的特征值是 $A A^T$ 或 $A^T A$ 的

$A^T A$ 是半正定矩阵 positive semi-definite matrix，特征值都是非负

掌握SVD的计算

5.3 SVD的应用

图像压缩：保留前k个最大奇异值

数据降维

去噪

推荐系统

潜在语义分析

伪逆

角度	SVD	PCA
数学本质 Mathematical essence	分解任意矩阵 ($A=U\Sigma V^T$)	对协方差矩阵做特征值分解
适用对象 Applicable objects	任意形状矩阵, 包括非方阵、稀疏矩阵	数值型数据矩阵, 需要中心化
计算过程 Calculation process	可直接分解数据矩阵	先算 covariance matrix, 再做 EVD
降维方式 Dimensionality reduction	保留前 (k) 个最大 singular values	保留前 (k) 个最大 eigenvalues 对应的 eigenvectors
物理意义 Physical meaning	揭示矩阵内部结构 intrinsic structure	找最大方差方向 maximum variance directions
稳定性 Stability	高维或稀疏数据更稳定	算协方差矩阵可能引入数值误差
应用 Use cases	推荐系统、LSA、图像压缩、信号去噪	数据可视化、特征降维、探索性分析
关系 Relationship	PCA 可以用 SVD 实现	PCA 是 SVD 在降维中的特殊应用